



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Maestría en Ingeniería de Software y Sistemas Informáticos

Actividad 2. Clasificación De Noticias Mediante Procesamiento De Lenguaje Natural

Fundamentos de inteligencia artificial

Estudiante: Miguel de Jesús Chávez Barragán

1. Introduccion.....	3
2. Metodo.....	3
2.1 Dataset.....	3
2.2 Construcción Del Texto De Entrada.....	4
2.3 Analysis Exploratorio.....	5
2.4 Preprocesamiento Del Texto.....	8
2.5 Representación Vectorial.....	9
Modelos Supervisados.....	10
3 Resultados.....	11
3.1 Métricas Globales.....	11
3.2 Reporte Por Clase.....	11
3.3 Matriz De Confusion.....	12
3.4 Validacion Cruzada.....	15
4 Análisis De Errores.....	15
6 Selección Del Modelo Final.....	16
7 Limitaciones.....	16
8 Mejoras Futuras.....	17
9 Conclusiones.....	17
10 Referencias.....	18

1. Introduccion

La clasificación automática de texto consiste en asignar una etiqueta a un documento a partir de su contenido. En este trabajo, cada documento es una noticia periodística y la etiqueta corresponde a una categoría temática. Este tipo de problema es común en el procesamiento de lenguaje natural, área que estudia métodos computacionales para analizar y representar lenguaje humano (Bird et al., 2009).

El problema no se resuelve directamente con texto crudo. Los modelos supervisados necesitan variables numéricas. Por eso, el flujo de trabajo transforma cada noticia en una representación vectorial antes de entrenar los modelos. En este caso se usó TF-IDF, sigla de term frequency-inverse document frequency. TF-IDF asigna peso a una palabra según su frecuencia en un documento y según qué tan común o rara es en el conjunto completo de documentos (Manning et al., 2008; Salton & Buckley, 1988).

El objetivo de la actividad fue desarrollar y comparar varios modelos supervisados capaces de clasificar noticias de AG News en cuatro categorías. La comparación se realizó con métricas de accuracy, precisión, recall y F1-score, además de validación cruzada, matriz de confusión y análisis de errores.

2. Metodo

2.1 Dataset

Se utilizó el dataset AG News Classification, indicado en la actividad como fuente de datos. Este conjunto contiene noticias clasificadas en cuatro categorías: World, Sports, Business y Science/Technology. AG News también ha sido usado como referencia en trabajos de clasificación de texto, por ejemplo en la evaluación de redes convolucionales a nivel de carácter de Zhang et al. (2015).

Tabla 1

Categorías del problema

Etiqueta	Clase usada en el notebook	Descripción breve
1	World	Noticias internacionales y política global.
2	Sports	Noticias deportivas.
3	Business	Economía, empresas, mercado y finanzas.
4	Sci/Tech	Ciencia, tecnología, productos y desarrollo técnico.

El dataset se recibió separado en dos archivos: train.csv y test.csv. El primero se usó para entrenar los modelos y el segundo para evaluar el rendimiento final.

Tabla 2

Estructura del dataset

Conjunto	Registros	Columnas originales	Uso
Entrenamiento	120000	Class Index, Title, Description	Ajuste de TF-IDF y entrenamiento de modelos.
Prueba	7600	Class Index, Title, Description	Evaluación final del rendimiento.

No se detectaron valores nulos en las columnas originales. Aun así, al construir la columna textual se usó `fillna("")` para evitar errores si el dataset cambiará en otra versión.

2.2 Construcción Del Texto De Entrada

El texto de entrada se construyó uniendo el título y la descripción de cada noticia:

```
train_df["text"] = train_df[TEXT_COL_1].fillna("") + " " +
train_df[TEXT_COL_2].fillna("")
test_df["text"] = test_df[TEXT_COL_1].fillna("") + " " +
test_df[TEXT_COL_2].fillna("")
Run
```

Esta decisión fue dada a que el título suele contener la idea central y la descripción agrega contexto. Usar ambos campos aumenta la información disponible para el vectorizador.

También se agrego un mapa de clases para que las tablas y gráficas fueran legibles:

```
CLASS_NAMES = {  
    1: "World",  
    2: "Sports",  
    3: "Business",  
    4: "Sci/Tech"  
}  
CLASS_ORDER = [CLASS_NAMES[label] for label in sorted(CLASS_NAMES)]
```

2.3 Analysis Exploratorio

El primer punto revisado fue el balance de clases. El balance es importante porque un conjunto desbalanceado puede hacer que la accuracy parezca buena aunque el modelo ignore clases minoritarias.

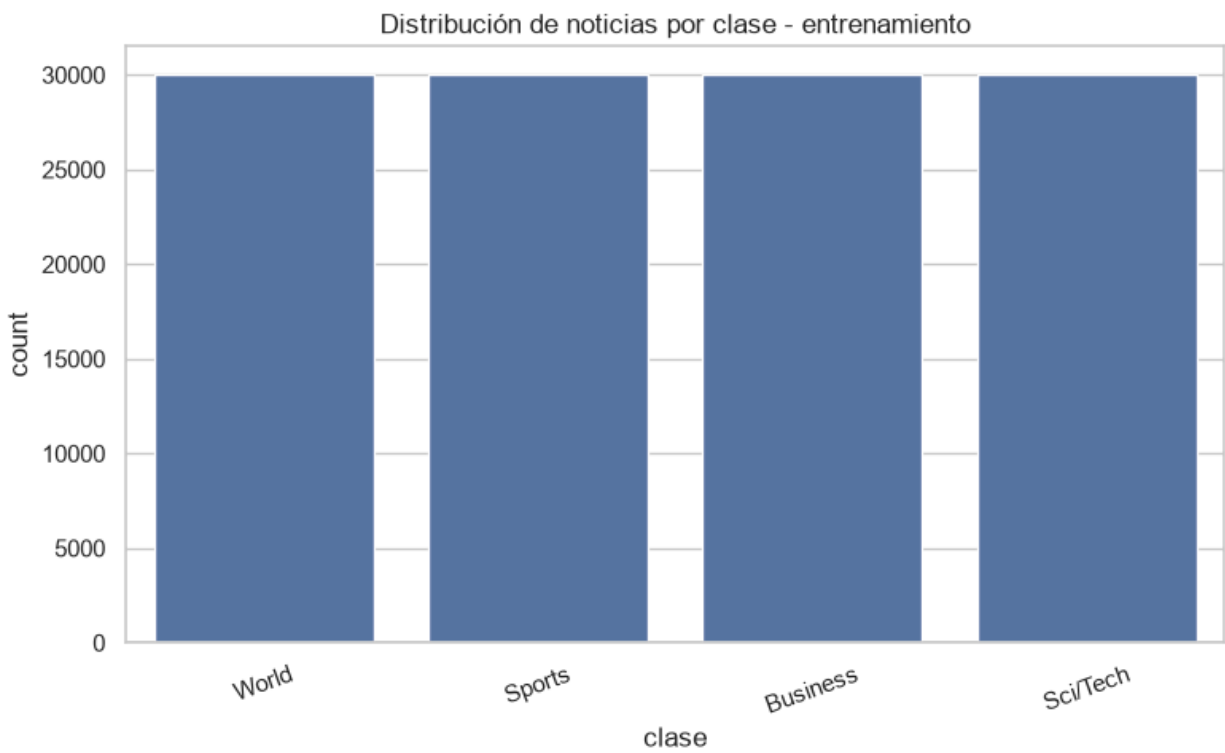
Tabla 3

Distribución por clase

Clase	Entrenamiento	Prueba
World	30000	1900
Sports	30000	1900
Business	30000	1900
Sci/Tech	30000	1900

Figura 1

Distribución de noticias por clase en entrenamiento

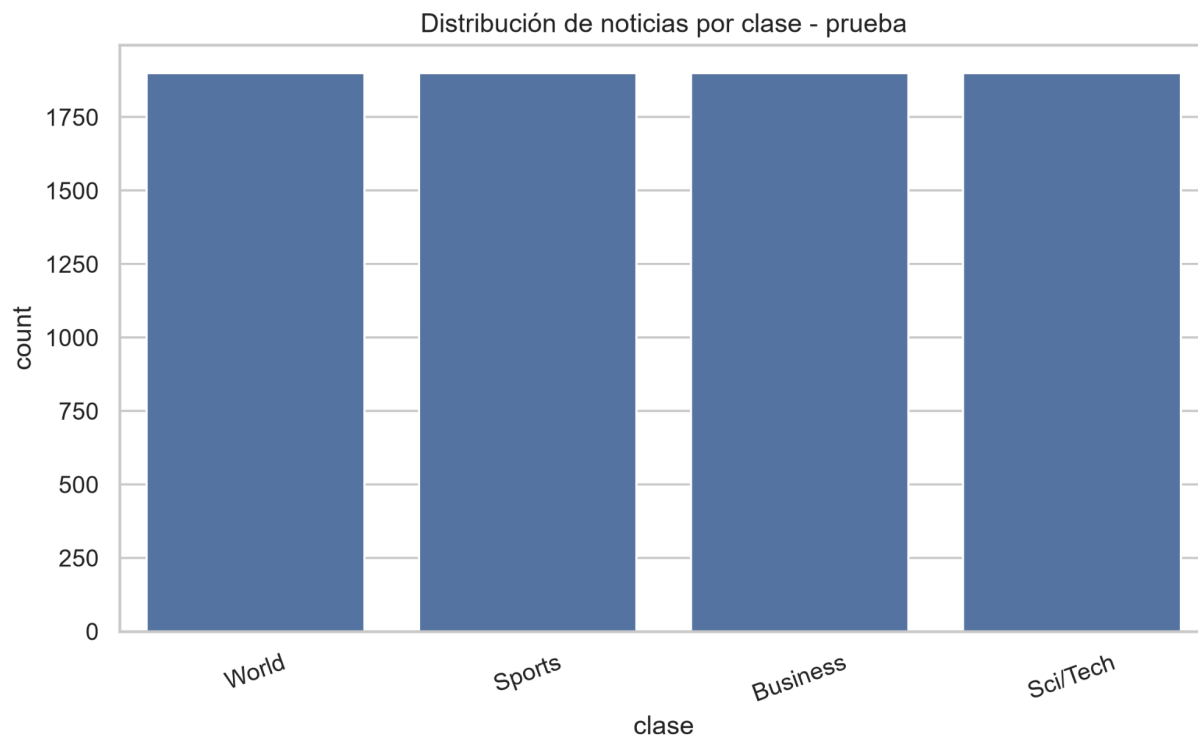


Distribución de noticias por clase en entrenamiento

Nota. La figura muestra que las cuatro clases tienen 30000 ejemplos en entrenamiento.

Figura 2

Distribución de noticias por clase en prueba



distribucion_clases_test.png

Nota. La figura muestra 1900 noticias por clase en el conjunto de prueba.

También se revisó la longitud del texto combinado. La longitud media fue casi igual entre entrenamiento y prueba, lo cual sugiere que ambos conjuntos tienen formatos comparables.

Tabla 4

Longitud del texto combinado

Estadístico	Entrenamiento	Prueba
Media	236.46	235.29
Desviacion estandar	66.53	65.30
Mediana	232	231
Maximo	1012	892

2.4 Preprocesamiento Del Texto

El preprocesamiento preparó el texto para convertirlo en variables numéricas. Primero se convirtió todo a minúsculas. Después se eliminaron caracteres no alfabéticos, se separó el texto en tokens, se quitaron stopwords y se aplicó lematización.

Un token es una unidad mínima de procesamiento textual, normalmente una palabra o signo separado por el tokenizador. La tokenización es necesaria porque muchas técnicas de PLN operan sobre unidades discretas del texto (Bird et al., 2009). Las stopwords son palabras muy frecuentes, como artículos o preposiciones, que pueden aportar poco al tema de una noticia. La lematización reduce una palabra a una forma base, por ejemplo plural a singular o variantes flexionadas a una forma común.

El código usado fue:

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"^[a-zA-Z\s]", " ", text)
    tokens = word_tokenize(text)
    tokens = [token for token in tokens if token not in stop_words
and len(token) > 2]
    tokens = [lemmatizer.lemmatize(token) for token in tokens]
    return " ".join(tokens)
```

El resultado de esta función es una versión más compacta del texto. Por ejemplo, una noticia sobre mercados y petróleo queda reducida a términos como oil, economy, stock, outlook y reuters. La limpieza ayuda a reducir ruido, aunque también tiene una limitación: al eliminar números se pueden perder señales útiles en noticias financieras o tecnológicas, como porcentajes, años, versiones de productos o precios.

2.5 Representación Vectorial

Los modelos supervisados de scikit-learn no entrenan directamente sobre cadenas de texto. Primero se requiere convertir cada noticia en un vector numérico. Para esto se usó TF-IDF.

TF-IDF combina dos ideas. La frecuencia de término mide cuantas veces aparece una palabra en un documento. La frecuencia inversa de documento reduce el peso de palabras que aparecen en demasiados documentos, porque suelen ser menos distintivas (Manning et al., 2008). Este enfoque ha sido clásico en recuperación de información y clasificación de texto (Salton & Buckley, 1988).

La configuración fue:

```
tfidf = TfidfVectorizer(  
    max_features=20000,  
    ngram_range=(1, 2),  
    min_df=3,  
    max_df=0.95  
)
```

Tabla 5

Parametros de TF-IDF

Parametro	Valor	Function
max_features	20000	Limita el vocabulario para controlar memoria y ruido.
ngram_range	(1, 2)	Usa palabras individuales y pares de palabras.
min_df	3	Descarta términos presentes en menos de tres documentos.
max_df	0.95	Descarta términos presentes en más del 95% de documentos.

Un n-grama es una secuencia de n elementos consecutivos. En este caso, unigrama significa una palabra individual y bigrama significa una secuencia de dos palabras. Los bigramas ayudan a capturar expresiones como stock market o world cup.

Después de vectorizar, las matrices quedaron así:

Tabla 6

Dimensions de las matrices TF-IDF

Matriz	Forma
X_train_tfidf	(120000, 20000)
X_test_tfidf	(7600, 20000)

2.7 Modelos Supervisados

Se entrenaron tres modelos con la misma representación TF-IDF:

1. MultinomialNB.
2. LogisticRegression.
3. LinearSVC.

Naive Bayes funciona como una línea base rápida para texto. Regresión logística es un modelo lineal usado con frecuencia cuando las variables son pesos numéricos. LinearSVC implementa una máquina de vectores de soporte lineal; las máquinas de vectores de soporte buscan separar clases mediante márgenes amplios, idea introducida formalmente por Cortes y Vapnik (1995). La implementación se realizó con scikit-learn, una biblioteca de aprendizaje automático documentada por Pedregosa et al. (2011).

La evaluación incluyó validación cruzada. En este contexto, validación cruzada significa dividir el conjunto de entrenamiento en particiones, entrenar varias veces y medir si el rendimiento se mantiene estable. Se usó Pipeline para que TF-IDF se ajustara dentro de cada fold y no filtrara información.

3 Resultados

3.1 Métricas Globales

Se calcularon accuracy, precisión, recall y F1-score. Accuracy mide la proporción total de aciertos. Precisión mide qué proporción de las predicciones positivas de una clase fueron correctas. Recall mide qué proporción de los casos reales de una clase fueron recuperados. F1-score combina precisión y recall en una sola medida, útil cuando interesa equilibrar ambos tipos de error.

Tabla 7

Métricas en conjunto de prueba

Modelo	Accuracy	Precision	Recall	F1-score
Naive Bayes	0.900263	0.899927	0.900263	0.899832
Logistic Regression	0.913684	0.913537	0.913684	0.913513
LinearSVC	0.918289	0.918251	0.918289	0.918199

LinearSVC fue el mejor modelo en el conjunto de prueba. Su ventaja sobre Logistic Regression fue pequeña, pero parece que fue la mejor en todas las métricas.

3.2 Reporte Por Clase

El mejor modelo se elige automáticamente a partir del F1-score:

```
predictions = {  
    "Naive Bayes": nb_pred,  
    "Logistic Regression": lr_pred,  
    "LinearSVC": svm_pred,  
}  
  
best_model_name = results_df.sort_values("f1_score",  
ascending=False).iloc[0]["modelo"]  
best_pred = predictions[best_model_name]
```

Tabla 8

Reporte por clase para LinearSVC

Clase	Precision	Recall	F1-score	Support
World	0.93	0.91	0.92	1900
Sports	0.96	0.98	0.97	1900
Business	0.88	0.89	0.88	1900
Sci/Tech	0.90	0.90	0.90	1900

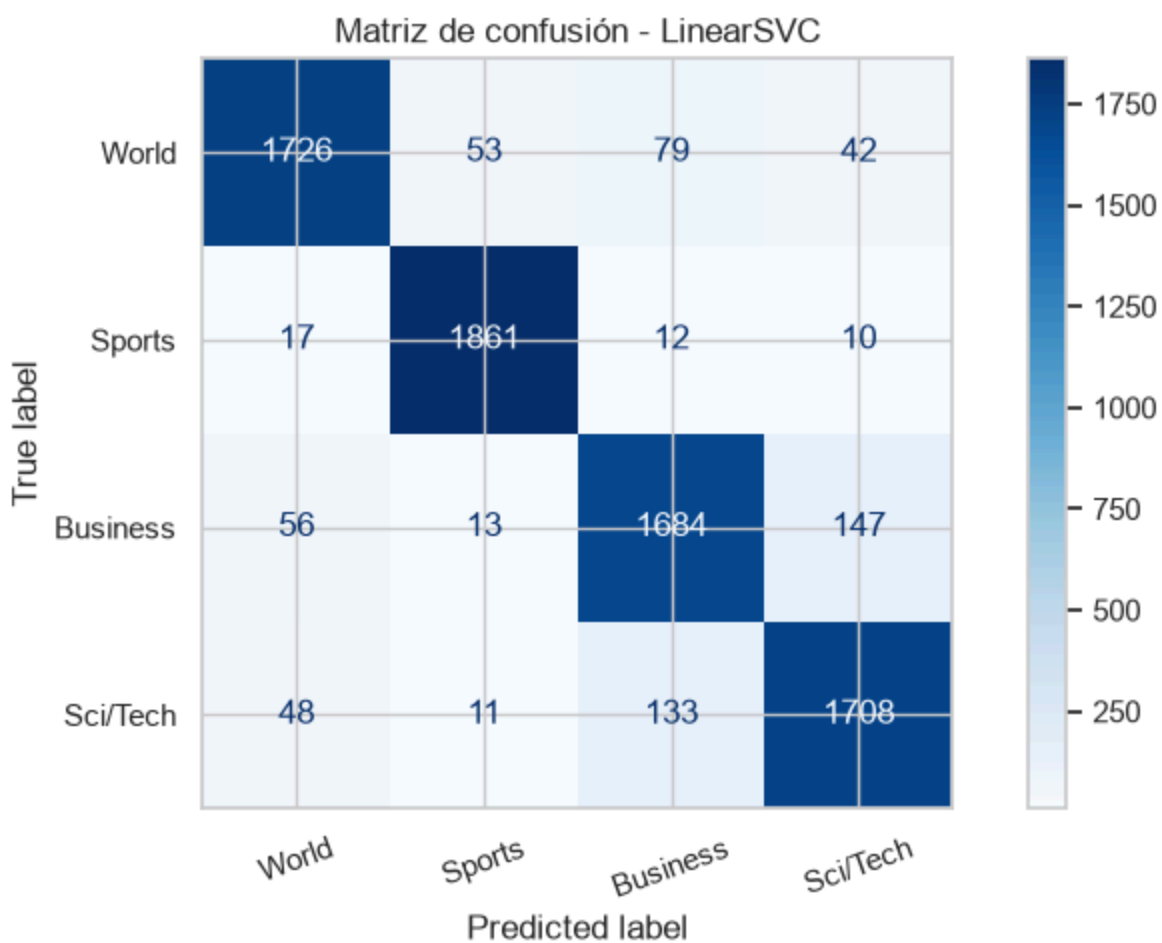
Sports fue la clase con mejor resultado, con recall de 0.98 y F1-score de 0.97. Business fue la clase más difícil, con F1-score de 0.88.

3.3 Matriz De Confusion

La matriz de confusión muestra cómo se distribuyen los aciertos y errores entre clases. Cada fila corresponde a la clase real y cada columna a la clase predicha.

Figura 3

Matriz de confusion de LinearSVC



Nota. Los valores de la diagonal son aciertos. Los valores fuera de la diagonal son errores.

Tabla 9

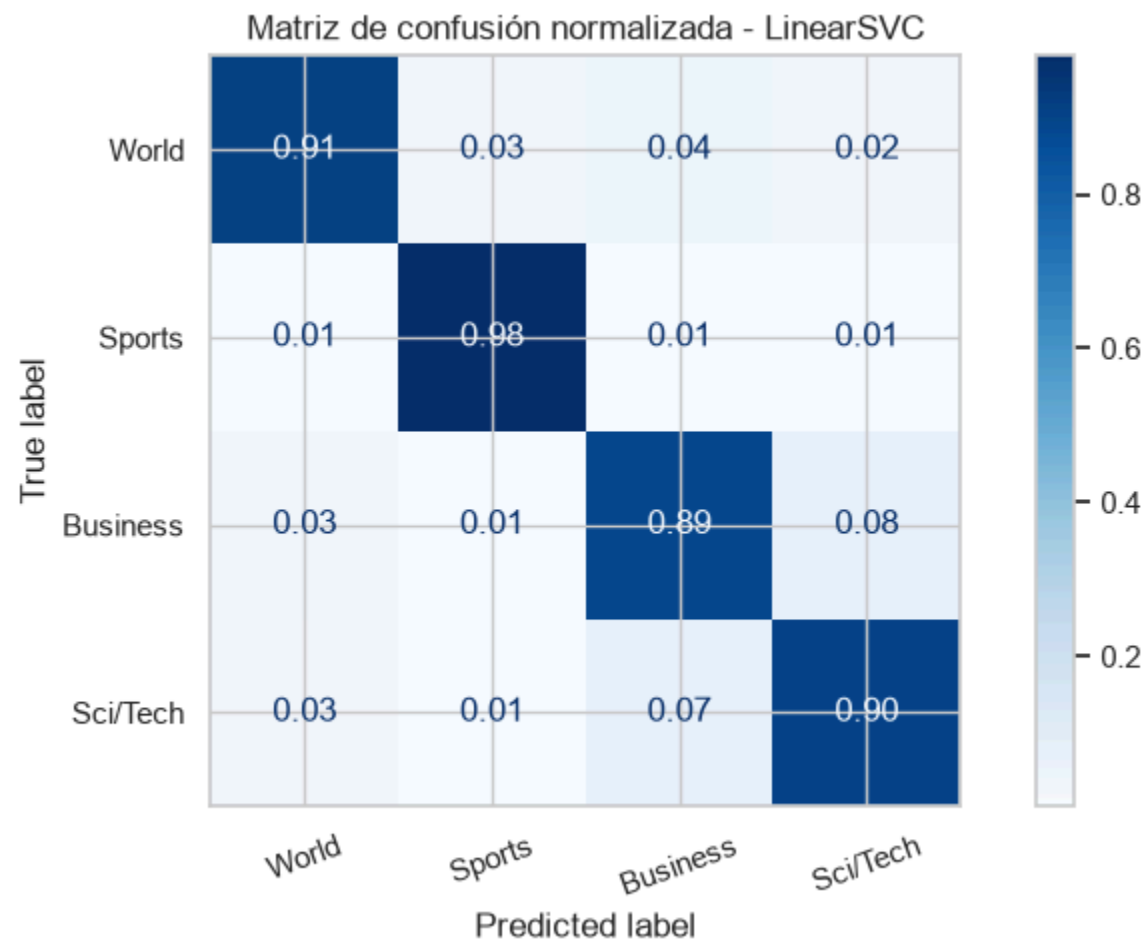
Matriz de confusión con conteos

Clase real	World	Sports	Business	Sci/Tech	Aciertos
World	1726	53	79	42	1726
Sports	17	1861	12	10	1861
Business	56	13	1684	147	1684
Sci/Tech	48	11	133	1708	1708

El modelo acertó 6979 de 7600 noticias. La mayor confusión se dio entre Business y Sci/Tech: 147 noticias de Business fueron clasificadas como Sci/Tech, y 133 noticias de Sci/Tech fueron clasificadas como Business.

Figura 4

Matriz de confusion normalizada de LinearSVC



Nota. La matriz normalizada muestra proporciones por clase real. Por eso facilita comparar el recall entre clases.

La matriz normalizada confirma que Sports fue la categoría más estable. Business y Sci/Tech quedaron cerca, pero concentran la mayor parte de las confusiones relevantes.

3.4 Validacion Cruzada

Tabla 10

Validación cruzada con F1 ponderado

Modelo	CV F1 promedio	Desviacion estandar
Naive Bayes	0.897162	0.008342
Logistic Regression	0.903738	0.007320
LinearSVC	0.897629	0.008769

Este resultado es importante. LinearSVC fue mejor en el conjunto de prueba, pero Logistic Regression obtuvo el mejor F1 promedio en validación cruzada. Por eso, la selección final no debe presentarse como una victoria absoluta de LinearSVC. La diferencia real es más matizada.

4. Análisis De Errores

El notebook generó una tabla de errores con texto original, texto limpio, clase real y clase predicha. Mantener el texto original fue útil porque permite entender mejor la noticia. El texto limpio muestra que recibió el modelo, pero el texto original muestra el contexto.

Tabla 11

Ejemplos de errores de clasificación

Caso	Clase real	Clase predicha	Lectura
HP, servidores y ganancias	Sci/Tech	Business	Tecnología descrita con vocabulario financiero.
Google IPO	Sci/Tech	Business	Google apunta a tecnología, pero IPO apunta a negocios.
Chavez, Venezuela y	World	Business	Política internacional mezclada con petróleo y mercado.

Caso	Clase real	Clase predicha	Lectura
mercado petrolero			
Intel y televisión de alta definición	Business	Sci/Tech	Empresa y producto tecnológico en la misma noticia.
Juegos olímpicos India/UAE	Sports	World	Deporte con fuerte contexto geográfico.

El patrón principal está en la frontera Business/Sci-Tech. Esto no parece un error trivial del modelo. Varias noticias mezclan empresa, mercado, producto y tecnología. En esos casos, incluso una lectura humana puede depender del criterio editorial usado por el dataset.

5. Selección Del Modelo Final

El modelo final seleccionado fue LinearSVC.

La razón principal es su rendimiento en el conjunto de prueba: accuracy de 0.9183 y F1-score ponderado de 0.9182. También mostró una diagonal fuerte en la matriz de confusión y un comportamiento razonable en todas las clases.

La segunda opción sería Logistic Regression. En validación cruzada, este modelo obtuvo el mejor F1 promedio. Por eso, Logistic Regression sería una opción defendible si el criterio principal fuera estabilidad interna. En esta actividad se eligió LinearSVC porque el conjunto de pruebas separado funcionó como evaluación final.

6. Limitaciones

El trabajo tiene algunas limitaciones. Primero, la limpieza elimina números, lo cual puede quitar información útil en noticias de negocios y tecnología. Segundo, TF-IDF representa documentos por pesos de términos, pero no entiende significado contextual. Dos noticias con palabras parecidas pueden recibir vectores parecidos aunque el sentido editorial sea distinto. Tercero, no se ajustaron hiperparametros de manera exhaustiva. Por último, no se probaron embeddings ni modelos basados en transformers.

Un embedding es una representación densa de una palabra o documento en un espacio vectorial, donde palabras semánticamente cercanas tienden a quedar cerca entre sí (Mikolov et al., 2013). Un transformer es una arquitectura de aprendizaje profundo basada en mecanismos de atención, diseñada para modelar relaciones entre elementos de una secuencia sin depender de la recurrencia (Vaswani et al., 2017). Ambas alternativas podrían capturar relaciones semánticas más complejas que TF-IDF.

7. Mejoras Futuras

Para mejorar el flujo, convendría probar una limpieza menos agresiva que conserve números, porcentajes y símbolos financieros. También sería útil ajustar hiperparámetros de Logistic Regression y LinearSVC, sobre todo el parámetro C. Otra mejora sería comparar TF-IDF con embeddings pre entrenados o modelos transformer. Finalmente, se podría guardar el modelo final con Joblib para utilizarlo sin volver a entrenar.

9. Conclusiones

El flujo implementado cumple con los requisitos de la actividad. Se realizó exploración del dataset, preprocesamiento textual, representación vectorial, entrenamiento de tres modelos, validación cruzada, cálculo de matrices, matriz de confusión y análisis de errores.

El dataset está balanceado, lo que facilita una comparación justa entre modelos. LinearSVC fue el mejor modelo en el conjunto de prueba, con accuracy de 0.9183 y F1-score ponderado de 0.9182. Sports fue la clase más fácil de clasificar, mientras que Business y Sci/Tech concentraron la mayor confusión.

La lectura final debe ser cuidadosa: LinearSVC gana en prueba, pero Logistic Regression muestra mejor F1 promedio en validación cruzada. En términos prácticos, LinearSVC es una buena selección final para esta entrega, siempre que se reconozca que la diferencia con Logistic Regression es reducida.

9. Referencias

Anand, A. (s. f.). *AG News classification dataset*. Kaggle.

<https://www.kaggle.com/datasets/amananandrai/ag-news-classification-dataset>

Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.

<https://www.nltk.org/book/>

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273-297.

<https://doi.org/10.1007/BF00994018>

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*.

Cambridge University Press. <https://nlp.stanford.edu/IR-book/>

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space*. arXiv. <https://arxiv.org/abs/1301.3781>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://jmlr.org/papers/v12/pedregosa11a.html>

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513-523.

[https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03762>

Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. *Advances in Neural Information Processing Systems*, 28.

<https://arxiv.org/abs/1509.01626>